

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 — ARCHITECTURE DESIGN REPORT

Smart Building Occupancy Prediction and Space Utilization Platform

Field	Details
Course Code / Title	CSA1013 — Software Engineering
CO Assessed	CO2 — Requirements analysis, scalable architecture design, and feature prioritization
Assigned Problem	Smart Building Occupancy Prediction and Space Utilization Platform
Student Name	Ch.B.Vinay
Registration Number	192425148
Date	05/08/2026

Code of Conduct

I Ch.B.Vinay (192425148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Ch.B.Vinay*

1. Analysis of System Requirements

1.1 System Objectives

The problem statement identifies that large commercial buildings, educational institutions, and corporate offices suffer from inefficient utilization of meeting rooms, classrooms, and office spaces because manual monitoring cannot provide accurate, real-time occupancy information. This leads to unnecessary energy consumption and higher operational costs. The proposed platform must therefore collect data from IoT occupancy sensors, access control systems, and scheduling platforms; predict space usage; optimize room allocation; and reduce energy waste — ultimately improving facility management decisions.

1.2 Functional Requirements

The functional requirements below were derived directly from the stated objectives (monitoring, prediction, optimization, energy reduction, analytics, HVAC control, and facility planning):

ID	Functional Requirement
FR-1	Collect real-time occupancy data from IoT sensors (PIR, CO2, camera-based counters) installed across rooms and zones.
FR-2	Integrate with access control systems (badge/RFID/turnstile) to cross-validate entry and exit events.
FR-3	Integrate with scheduling/room-booking platforms to compare planned vs. actual utilization.
FR-4	Predict future occupancy trends using historical and real-time data through machine learning models.
FR-5	Recommend and automate optimal room allocation based on predicted demand and available capacity.
FR-6	Trigger smart HVAC and lighting control commands to the Building Management System (BMS) based on occupancy forecasts.
FR-7	Generate occupancy analytics dashboards and reports for facility managers.
FR-8	Send notifications/alerts to occupants and facility staff (e.g., room reassignment, underutilized space).

1.3 Non-Functional Requirements and Key Quality Attributes

Because the platform continuously ingests high-frequency sensor data and directly drives physical building controls (HVAC/lighting), its non-functional requirements are architecture-defining rather than incidental:

Quality Attribute	Description
Scalability	Must support thousands of concurrent sensor streams across multiple buildings/campuses without performance degradation.
Performance	Sensor-to-dashboard latency should remain low (near real-time, sub-second to few-second range) for occupancy updates.
Reliability	Predictions and control actions must remain accurate and available even if a subset of sensors/services fail.
Availability	Core monitoring and dashboard services should be available 24/7 (target 99.9% uptime) since HVAC control depends on it.

Quality Attribute	Description
Security	Sensor data, access-control logs, and building-control channels must be authenticated, encrypted, and access-controlled.
Maintainability	Components (ML models, integrations, dashboards) must be independently updatable without full system downtime.
Interoperability	Must integrate heterogeneous protocols (MQTT, BACnet, REST) from different sensor and BMS vendors.
Energy Efficiency	The platform itself, and the decisions it drives, should minimize unnecessary HVAC/lighting energy usage.

2. Selection of Suitable Software Architecture

2.1 Selected Architectural Style

Hybrid Architecture: Event-Driven Microservices with an Edge Computing Layer.

The platform combines three complementary styles: (1) an Edge Computing layer at sensor/gateway level to normalize protocols and pre-process data close to the source; (2) an Event-Driven Architecture, using a publish–subscribe message broker, to handle high-volume, asynchronous, real-time sensor streams; and (3) a Microservices Architecture for the cloud-side business logic (prediction, allocation, energy control, analytics, notifications), each independently deployable and scalable.

2.2 Justification

- High-volume, continuous IoT streams from many sensors are naturally asynchronous and bursty — an event-driven backbone (message broker) absorbs load spikes and decouples producers from consumers, which a purely synchronous request/response style (e.g., plain client-server) would not handle well.
- The system has clearly separable, independently evolving concerns — occupancy prediction (ML), room allocation, HVAC control, analytics, and notifications — each with different scaling and update needs, making microservices more appropriate than a single monolithic or layered application.
- Edge pre-processing reduces bandwidth/latency and allows the system to keep functioning locally (e.g., basic occupancy counts) even during temporary cloud connectivity loss, improving reliability for a building-control use case.
- A pure Serverless or purely Service-Oriented (SOA) approach was considered but rejected as the sole style: Serverless alone struggles with the persistent, high-frequency streaming ingestion required here, and traditional SOA's heavier enterprise-bus overhead is unnecessary given the platform's more focused, cloud-native scope. Elements of both (managed event services, well-defined service contracts) are still used within the hybrid design.

This hybrid approach directly supports the required quality attributes — scalability, low-latency performance, reliability, availability, security, and maintainability — while remaining flexible enough to add new sensor types, buildings, or analytics features over time.

3. Software Architecture Diagram

The diagram below illustrates the end-to-end structure of the platform: data sources and edge processing, cloud ingestion/messaging, storage, the microservices business-logic layer, and the application/presentation layer, along with the closed-loop control path back to the Building Management System (BMS).

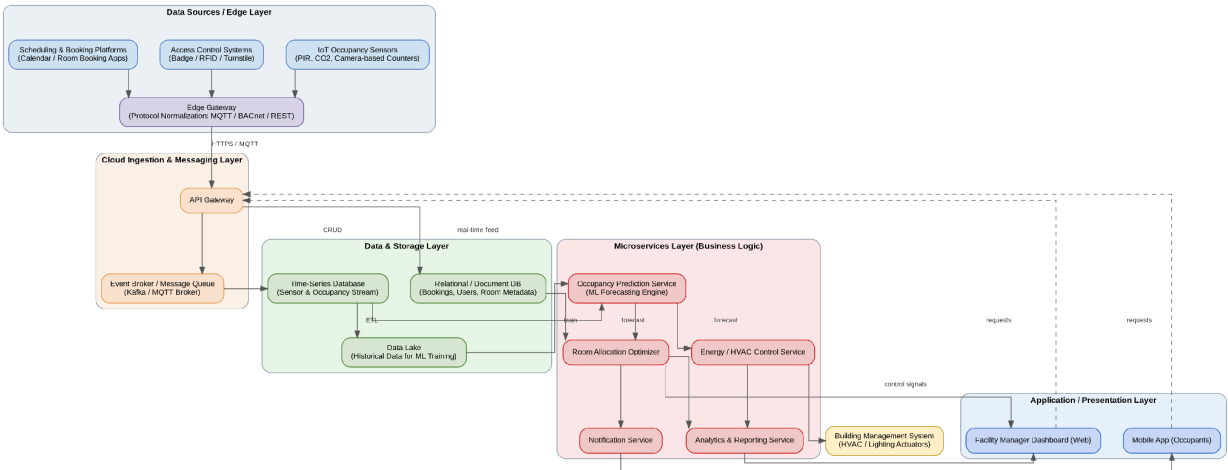


Figure 1: Layered event-driven microservices architecture for the Smart Building Occupancy Prediction and Space Utilization Platform.

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Purpose / Responsibility
IoT Occupancy Sensors & Access Control	Capture raw motion/CO2/count and badge-in/out events at the edge; the primary data sources for the platform.
Scheduling/Booking Platforms	External systems supplying planned bookings, used to compare intended vs. actual space use.
Edge Gateway	Normalizes heterogeneous sensor protocols (MQTT, BACnet, vendor REST) into a uniform event format before transmission to the cloud.
API Gateway	Single secured entry point for all inbound/outbound traffic; handles authentication, rate limiting, and routing to backend microservices.
Event Broker / Message Queue	Decouples data producers (sensors) from consumers (services) via publish–subscribe messaging (e.g., Kafka/MQTT broker), enabling asynchronous, event-driven processing.
Time-Series Database	Stores high-frequency occupancy/sensor readings optimized for time-based queries and streaming analytics.
Relational/Document Database	Stores structured data — bookings, users, room metadata, configuration.
Data Lake	Stores raw historical data used to train and retrain machine learning models.
Occupancy Prediction Service	Core ML microservice that forecasts short- and long-term occupancy per room/zone using historical and streaming data.
Room Allocation Optimizer	Consumes predictions to recommend/automate optimal room and space assignment.

Component	Purpose / Responsibility
Energy/HVAC Control Service	Translates occupancy forecasts into HVAC/lighting control signals sent to the Building Management System.
Analytics & Reporting Service	Aggregates data for dashboards, utilization reports, and trend analysis.
Notification Service	Delivers alerts (e.g., room changes, underutilization) to facility staff and occupants.
Facility Manager Dashboard & Mobile App	Presentation layer for facility managers and occupants to view analytics, receive alerts, and interact with bookings.
Building Management System (BMS)	External actuation system that physically controls HVAC and lighting based on commands received.

4.2 Communication and Data Flow

- IoT occupancy sensors, access-control devices, and scheduling platforms send raw events to the Edge Gateway, which normalizes protocols (MQTT/BACnet/REST) into a common event format.
- The Edge Gateway forwards normalized events over HTTPS/MQTT to the API Gateway, which authenticates requests and routes streaming data into the Event Broker (publish–subscribe) and structured data (bookings, users) into the relational/document database.
- The Event Broker streams occupancy events into the Time-Series Database in real time and also feeds them directly to the Occupancy Prediction Service; historical data is periodically moved from the time-series store into the Data Lake to retrain ML models.
- The Occupancy Prediction Service publishes forecasts consumed by the Room Allocation Optimizer and the Energy/HVAC Control Service.
- The Room Allocation Optimizer and Energy/HVAC Control Service feed the Analytics & Reporting Service and the Notification Service, and the Energy/HVAC Control Service sends control signals to the Building Management System to adjust HVAC/lighting.
- Facility managers and occupants interact through the Dashboard and Mobile App, which call back through the API Gateway for on-demand queries and receive analytics/notifications, closing the loop between prediction, action, and human oversight.

5. Discussion of Quality Attributes

Attribute	How the Architecture Addresses It
Scalability	Event-driven microservices allow each service (prediction, allocation, analytics) to be scaled independently. The message broker buffers load spikes from thousands of sensors, and horizontal scaling of consumers handles growth across multiple buildings.
Security	The API Gateway centralizes authentication (OAuth2/JWT), TLS encryption secures sensor-to-cloud and control-channel traffic, and role-based access control restricts who can view analytics or issue BMS commands.
Performance	The time-series database and in-memory event streaming keep sensor-to-dashboard latency low; edge gateways pre-filter/aggregate data to reduce unnecessary cloud round-trips.
Reliability	The message broker persists events so no sensor reading is lost if a downstream service is briefly unavailable; services are stateless and can be restarted/replaced without data loss.

Attribute	How the Architecture Addresses It
Maintainability	Loose coupling between microservices (via well-defined events/APIs) allows the ML model, allocation logic, or dashboard to be updated or redeployed independently.
Availability	Redundant service instances behind the API Gateway, combined with database replication, support the 24/7 availability required for continuous HVAC/lighting control.

6. Justification of Architectural Decisions

6.1 Rationale

The core rationale is that occupancy prediction and building control are inherently real-time, data-intensive, and multi-source problems. An event-driven backbone was chosen specifically because it matches the natural shape of the workload (continuous, asynchronous sensor streams) rather than forcing it into a request/response model. Microservices were chosen so that the ML-heavy prediction component — which has very different scaling, deployment, and update cadence needs than, say, the notification service — can evolve and scale independently.

6.2 Trade-offs Considered

- Complexity vs. flexibility: A monolithic or simple layered architecture would be easier to build and deploy initially, but would not scale well across multiple buildings or allow independent updates to the ML model without redeploying the whole system. The added operational complexity of microservices (service discovery, distributed monitoring, inter-service contracts) was accepted in exchange for scalability and maintainability.
- Latency vs. centralization: Performing all processing centrally in the cloud would simplify the architecture but increase latency and bandwidth cost; introducing an edge layer adds deployment/maintenance overhead at each building but reduces latency and preserves basic functionality during connectivity loss.
- Consistency vs. availability: The event-driven design favors availability and responsiveness (eventual consistency between sensor state and dashboards) over strict immediate consistency, which is an acceptable trade-off for occupancy analytics but is compensated for critical control paths (HVAC commands) by using acknowledgement and retry mechanisms.

6.3 Support for Future Enhancements and Long-Term Maintainability

- New sensor types or vendors can be onboarded by adding an edge adapter and publishing to the existing event schema, without changing downstream services.
- Additional buildings/campuses can be added by scaling the messaging and storage layers horizontally, since services are stateless and independently deployable.
- New capabilities (e.g., predictive maintenance, occupant well-being analytics, integration with renewable-energy management) can be added as new microservices subscribing to the same event streams, without disrupting existing services.
- Independent versioning and deployment of each microservice supports continuous improvement of the ML prediction model over time as more historical data accumulates in the Data Lake.